Copyright All rights reserved

17. 전처리기와 빌드 제어
(Preprocessor and Build Control)

Heesung Oh
https://github.com/3dapi

17. 전처리기와 빌드 제어

Heesung Oh
Copyright All rights reserved

- 전처리기 (Preprocessor)
  - 컴파일 이전 단계에서 동작
  - 코드를 텍스트 수준에서 변형

- 주요 기능
  - #include → 파일 삽입
  - #define → 치환
  - #if → 조건 분기

→ 실행 코드가 아니라 빌드 제어 도구

17.1 Directive와 명령문의 구조적 차이

Heesung Oh
Copyright All rights reserved

- Directive
  - 컴파일 이전 처리
  - 문법 검사 없음
  - 코드 생성 없음

- C 명령문
  - 컴파일 단계 처리
  - 문법 검사 있음
  - 실행 코드 생성

→ 전처리기는 **코드 생성 전 단계**

17.2 매크로: 빌드 시점 코드 생성

Heesung Oh
Copyright All rights reserved

- 매크로 (Macro)
  - 반복되는 코드 패턴 정의
  - 전처리 단계에서 코드 확장
  - 코드 재사용
  - 수정 비용 감소

- 특징
  - 함수 호출 아님
  - 텍스트 치환 기반

17.2.1 반복 코드의 통합 관리

Heesung Oh
Copyright All rights reserved

- 반복 코드 문제
  - 동일 코드가 여러 곳에 존재
  - 수정 시 모든 위치 변경 필요

- 매크로 효과
  - 한 곳에서 관리
  - 전체 코드 일관성 유지

```
#define LOG_INFO(msg) \
 printf("[INFO] (%s:%d) %s\n", __FILE__, __LINE__, (msg))
```

```
printf("[INFO] %s\n", ("start"));
printf("[INFO] %s\n", ("processing"));
printf("[INFO] %s\n", ("end"));
```

17.2.1 반복 코드의 통합 관리

Heesung Oh
Copyright All rights reserved

- C 표준 매크로

매크로 이름	내용	예시 값
__FILE__	현재 소스 파일 이름	"main.c"
__LINE__	현재 코드의 줄 번호	42
__DATE__	컴파일이 수행된 날짜	"Mar 22 2026"
__TIME__	컴파일이 수행된 시간	"14:35:10"
__STDC__	표준 C 구현 여부 (1이면 표준)	1
__STDC_VERSION__	사용 중인 C 표준 버전	201112L, 202311L

- __func__, __FUNCTION__ (MSVC)
  - 식별자, 함수 내부에서만 유효, 해당 함수의 이름을 문자열로 반환

```
void test(void)
{
    printf("function: %s\n", __func__);
}
```

17.2.1 반복 코드의 통합 관리

Heesung Oh
Copyright All rights reserved

- **매크로 예제**

```
void PrintMacroValue(void)
{
    printf("__FILE__: %s\n", __FILE__);
    printf("__LINE__: %d\n", __LINE__);
    printf("__DATE__: %s\n", __DATE__);
    printf("__TIME__ : %s\n", __TIME__);
    // 일부 컴파일러(MSVC)에서 __STDC__ 지원 안함.
    //printf("__STDC__ : %d\n", __STDC__);
    printf("__STDC_VERSION__ : %d\n", __STDC_VERSION__);
    printf("__func__: %s\n", __func__);
    printf("__FUNCTION__: %s\n", __FUNCTION__);
}
```

17.2.2 객체 매크로

Heesung Oh
Copyright All rights reserved

- 객체 매크로
  - 토큰을 다른 토큰으로 단순 치환
  - 타입 없음. 메모리 없음

- 사용 예)
  - 전처리 전

```
#define BUFFER_SIZE 256
int buf[BUFFER_SIZE];
```

  - 전처리 후

```
int buf[256];
```

17.2.3 함수형 매크로

Heesung Oh
Copyright All rights reserved

- 함수형 매크로
  - 함수 형태 매크로
  - 함수처럼 보이지만 치환
  - 전달된 식 그대로 확장

- 사용 예
  - 전처리 전

```
#define SQUARE(x) ((x) * (x))
```

```
int r = SQUARE(a + 1);
```

  - 전처리 후

```
int r = ((a + 1) * (a + 1));
```

17.2.3 함수형 매크로 (작성 규칙 / 세미콜론)

Heesung Oh
Copyright All rights reserved

- 괄호 필수
  - #define ADD(a,b) ((a)+(b))
  - 괄호 없으면 연산 우선순위 오류 발생

- 세미콜론 주의
  - 세미콜론도 함께 치환됨
  - Ex)
    - #define SIZE 100;
    - int arr[SIZE]; → int arr[100;];

→ 매크로 정의에는 **세미콜론 사용 안함**

17.2.4 매크로의 확장 기능: 문자열화와 토큰 결합

Heesung Oh
Copyright All rights reserved

- 단순 치환을 매크로 확장 기능
  - 문자열화 (#)
  - 토큰 결합 (##)

→ **코드 생성 도구**로 활용 가능

17.2.4.1 문자열화 연산자

Heesung Oh
Copyright All rights reserved

- **문자열화 (#)**

```
#define TO_STRING(x) #x

TO_STRING(hello)

// 전처리 결과:
"hello"
```

- **활용**
  - 디버그 출력
  - 로그 메시지

```
#define PRINT_VAR(x) printf(#x " =
%d\n", x)

int value = 10;
PRINT_VAR(value);

// 전처리 후:
printf("value" " = %d\n", value);
```

17.2.4.2 토큰 결합 연산자

Heesung Oh
Copyright All rights reserved

- **토큰 결합 (##)**

```
#define CONCAT(a, b) a##b

CONCAT(my, Var)

//전처리 후:
myVar
```

- **활용**
  - **변수 이름 생성**

```
#define MAKE_VAR(name) int var_##name

MAKE_VAR(count);

// 전처리 결과:
int var_count;
```

17.2.5 매크로의 재확장과 중첩

Heesung Oh
Copyright All rights reserved

- **매크로 재 확장**
  - 매크로는 단계적으로 계속 확장
  - 매크로안의 매크로를 확장

```
#define VALUE 10
#define DOUBLE(x) ((x) * 2)
```

```
int r = DOUBLE(VALUE);
```

```
// 1단계:
DOUBLE(VALUE)
↓
((VALUE) * 2)

// 2단계:
((10) * 2)

// 최종 결과:
int r = ((10) * 2);
```

```
#define BASE 5
#define ADD_BASE(x) ((x) + BASE)

int v = ADD_BASE(3);

// 전처리 결과:
int v = ((3) + 5);
```

```
// 허용 안함
#define A B
#define B A
```

17.3 조건부 컴파일: 빌드 분기

Heesung Oh
Copyright All rights reserved

- 조건부 컴파일
  - 빌드 단계에서 코드 선택
  - 실행 중 분기 아님
  - 컴파일 전에 코드 포함 여부 결정

- 용도
  - Debug / Release
  - 기능 ON/OFF
  - 플랫폼 분기

17.3.1 조건부 컴파일-기본 구조

Heesung Oh
Copyright All rights reserved

- 조건부 컴파일 구조

```
#ifdef
#ifndef

#if
#elif
#else
#endif
```

→ 조건에 따라 코드 포함/제거

17.3.1 조건부 컴파일-기본 구조 (#ifdef/ifndef)

Heesung Oh
Copyright All rights reserved

- 기본 사용
  - #ifdef
  - #ifndef
  - 정의 여부 / 값 기준 분기
  - → 조건 만족 시 코드 포함

```
#ifdef _DEBUG
    printf("debug\n");
#endif
```

```
#ifndef NDEBUG
    printf("디버그 모드\n");
#endif
```

```
#define USE_NETWORK 1

#if USE_NETWORK
    init_network();
#endif
```

```
#ifndef MY_HEADER_H
#define MY_HEADER_H

// 내용

#endif
```

17.3.1 조건부 컴파일-기본 구조 (defined/undef)

Heesung Oh
Copyright All rights reserved

- defined 연산자
  - #if defined(_DEBUG)
  - 복합 조건 가능
    - && || 사용
  - → 유연한 조건 구성

```
#if defined(_DEBUG)
    printf("debug\n");
#endif

//또는
#if defined(_DEBUG) &&
    defined(USE_NETWORK)
    printf("debug network\n");
#endif
```

- #undef
  - 매크로 해제
  - → 다시 정의 가능
  - → 설정 변경 시 사용

```
#define MODE 1

#undef MODE
#define MODE 2
```

17.3.1 조건부 컴파일-기본 구조 (#elif / #else)

Heesung Oh
Copyright All rights reserved

- 다중 분기
  - 여러 조건 처리

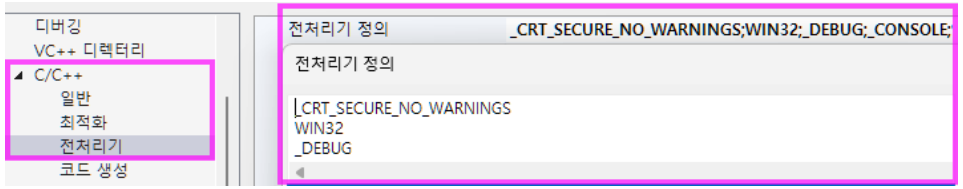
```
#define MODE 2

#if MODE == 1
    printf("Mode 1\n");
#elif MODE == 2
    printf("Mode 2\n");
#else
    printf("Unknown Mode\n");
#endif
```

17.3.2 빌드 환경에서 매크로 정의

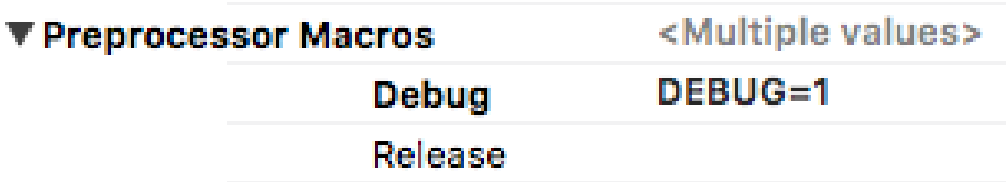
Heesung Oh
Copyright All rights reserved

- Visual Studio: _DEBUG / NDEBUG



- 안드로이드
  - LOCAL_CFLAGS += -DUSE_NETWORK
  - LOCAL_CFLAGS += -DDEBUG_MODE=1

- Xcode



- 공통 원리
  - -D 옵션
  - 빌드 구성(Debug/Release 등)에 따라 다른 매크로 집합
  - 소스 코드 수정 안하고 빌드 구조 제어

17.4 플랫폼 대응: 구현 선택 전략

Heesung Oh
Copyright All rights reserved

- 플랫폼 대응
  - 환경에 따라 코드 선택
  - Windows / Linux / Android
  - → 동일 인터페이스 유지
  - → 내부 구현만 변경

17.4.1 플랫폼 판별 매크로

Heesung Oh
Copyright All rights reserved

- 플랫폼 매크로
  - Windows → _WIN32, _WIN64
  - Linux → __linux__
  - Android → __ANDROID__
  - macOS → __APPLE__
  - iOS → __APPLE__ + TARGET_OS_IPHONE

- → 컴파일러가 자동 정의

17.4.2 동일 인터페이스, 다른 구현

Heesung Oh
Copyright All rights reserved

- 플랫폼마다 다른 코드
  - 빌드 시 하나만 선택
  - → 실행 비용 없음

```
#if defined(_WIN32)
  #include <windows.h>
  static inline void sleep_ms(unsigned ms)
  {
    Sleep(ms);
  }
#elif defined(__APPLE__) && defined(__MACH__)
  #include <TargetConditionals.h>
  #include <unistd.h>
  static inline void sleep_ms(unsigned ms)
  {
    usleep(ms * 1000);
  }
#elif defined(__ANDROID__) || defined(__linux__)
  #include <unistd.h>
  static inline void sleep_ms(unsigned ms)
  {
    usleep(ms * 1000);
  }
#else
  #error Unsupported platform
#endif
```

17.5 #include와 다중 파일 구조

Heesung Oh
Copyright All rights reserved

- #include
  - 파일 참조 아님
  - 파일 내용 삽입

  - → 코드가 그대로 복사됨

17.5.1 헤더와 소스의 역할 분리

Heesung Oh
Copyright All rights reserved

- 모듈 구조
  - .h → 선언
  - .c → 정의
- 목적
  - 코드 분리
  - 재사용

```c
// math.h
#pragma once
int add(int a, int b);
```

```c
// math.c
#include "math.h"

int add(int a, int b)
{
    return a + b;
}
```

```c
// main.c
#include "math.h"

int main(void)
{
    int r = add(1, 2);
    return r;
}
```

17.5.2 선언과 정의의 구분

Heesung Oh
Copyright All rights reserved

- 선언 vs 정의
  - 선언 → 존재 알림
  - 정의 → 실제 생성

- 선언은 여러 번 가능
- 정의는 하나만 가능

17.5.3 중복 정의와 링커 오류

Heesung Oh
Copyright All rights reserved

- 중복 정의 문제
  - 헤더에 정의 작성 시
  - 여러 파일에 복사됨
  - 링커 오류 발생

```
// config.h
#pragma once
int value = 0;  // 잘못된 예
```

17.5.4 extern과 단일 정의 원칙

Heesung Oh
Copyright All rights reserved

- 정의는 하나만 존재
  - extern 사용
  - 실제 정의가 하나만 존재
    → extern 은 여러 번 사용해도 문제 없음

```c
//config.h
#pragma once
extern int value;
```

```c
//config.c
#include "config.h"
int value = 0;
```

```c
//other.c
#include "config.h"
//중복 extern 선언
// 사용에 문제 없음
extern int value;
```

학습 정리

Heesung Oh
Copyright All rights reserved

- 전처리기
  - 텍스트 치환
  - 빌드 단계 제어

  - 매크로 → 코드 생성
  - 조건부 컴파일 → 코드 선택
  - #include → 파일 삽입
  - → 빌드 구조 이해가 핵심

실습 과제

Heesung Oh
Copyright All rights reserved

- 전처리 결과 확인하기
- 괄호 실수로 인한 오류 분석
- 조건부 컴파일 실험
- 플랫폼 분기 작성
- 중복 정의 오류 만들기
- include guard 실험